

Configuring Windows Server Core with PowerShell – Michael Waterman

 michaelwaterman.nl/2026/01/11/how-to-configuring-windows-server-core-with-powershell

Michael Waterman

January 11, 2026



A couple of weeks ago, I came across a discussion on Reddit about Windows Server Core versus Windows Server with a GUI. Not the usual debate about usability or learning curves, but something more telling, a lot of people were genuinely struggling to set up a Server Core installation from scratch.

Yes, there's the built-in `sconfig` utility. It works, and for a one-off setup it's perfectly fine. But let's be honest, if you're running Server Core in a production environment, clicking through menus or manually configuring systems shouldn't be part of the plan. Server Core practically *begs* for automation.

Not just during installation, but especially when it comes to base configuration: disk layout, IP addressing, DNS settings, firewall configuration, hostname, domain membership, all the boring, repeatable stuff that should be predictable and scriptable.

Every organization has its own standards and requirements, but that Reddit thread made me think:

why not share how I configure a Windows Server Core system using PowerShell only?

This post is exactly that. It's a *command-heavy, practical walkthrough* showing how to configure Windows Server Core using PowerShell, no GUI, no `sconfig`, no magic. Just commands, short explanations, and patterns you can reuse in your own automation. The goal isn't to provide a one-size-fits-all solution, but to give you a solid foundation you can adapt, script, and integrate into your deployment process. Let's dive in...

Scope & assumptions

This post assumes:

- A fresh Windows Server Core installation
- Local Administrator access
- PowerShell (no GUI, no `sconfig`)
- Commands are executed locally unless stated otherwise

The focus is on repeatable, automation-friendly configuration, not one-off manual changes.

Disk management

Before installing roles or joining a domain, I always make sure the disk layout is in a *known and predictable state*. Especially on Server Core, where you don't have a graphical safety net, getting this right upfront saves a lot of pain later. All examples below assume you are working on a fresh installation.

List all local disks

Start by identifying the available disks:

Tip! Pay close attention to the **Disk Number**. This is the identifier you'll use in all subsequent disk operations.

List all volumes

To see existing volumes and drive letters:

This is useful to verify:

- existing partitions
- current drive letters
- filesystem types

Change the drive letter of the CD-ROM/ DVD

I usually move the CD-ROM drive out of the way early to avoid conflicts with future data volumes.

```
Get-CimInstance -ClassName Win32_Volume -Filter "DriveType = 5" |  
Where-Object { $_.DriveLetter -ne 'X:' } |  
Set-CimInstance -Property @{ DriveLetter = 'X:' }
```

This ensures drive letters like D: remain available for data or application use.

Initialize a disk

If a disk is uninitialized, you'll need to initialize it before it can be used:

```
Initialize-Disk -Number <DiskNumber> -PartitionStyle GPT
```

GPT is the recommended partition style for modern Windows deployments.

Bring a disk online

Disks can be offline by default on fresh installs or SAN-attached storage:

```
Set-Disk -Number <DiskNumber> -IsOffline $false
```

Set a disk as writable

If a disk is marked as read-only, clear that flag:

```
Set-Disk -Number <DiskNumber> -IsReadOnly $false
```

Create a volume

Once the disk is ready, create a new volume:

```
New-Volume `  
-DiskNumber <DiskNumber> `  
-FriendlyName "<Label>" `  
-FileSystem NTFS `  
-DriveLetter "<DriveLetter>"
```

This command:

- creates the partition
- formats it
- assigns a drive letter
- sets a readable volume label

For most workloads, NTFS defaults are perfectly fine. If you have specific performance requirements, this is the point where you'd adjust allocation unit size or filesystem choice.

Get partition information

To inspect a specific partition:

```
Get-Partition -DiskNumber <DiskNumber> -PartitionNumber <PartitionNumber>
```

This can be useful when validating scripts or troubleshooting unexpected layouts.

Networking configuration

On Server Core, networking is one of the first things you want to get into a known-good state. A clean adapter name, a predictable IP configuration, and correct DNS behavior make everything else (domain join, updates, remote management) a lot less painful.

Rename the network adapter

I like to rename the active adapter to something meaningful (for example corp). This makes scripts easier to read and prevents mistakes when multiple NICs exist.

List adapters:

Rename the active adapter:

```
Get-NetAdapter | Where-Object Status -eq "Up" | Rename-NetAdapter -NewName "corp"
```

Set a static IPv4 address

Assign a static IP, prefix length, and default gateway:

```
New-NetIPAddress `
  -InterfaceAlias "corp" `
  -IPAddress 192.168.66.10 `
  -PrefixLength 24 `
  -DefaultGateway 192.168.66.1 `
  -AddressFamily IPv4
```

Set DNS Servers

Point the adapter to your DNS servers (typically [AD-integrated DNS](#)):

```
Set-DnsClientServerAddress `
  -InterfaceAlias "corp" `
  -ServerAddresses 192.168.66.11, 192.168.66.10
```

Set DNS client behavior

This configures the connection-specific DNS suffix and ensures the adapter registers itself in DNS (useful for domain-joined systems and predictable name resolution).

```
Set-DnsClient `
  -InterfaceAlias "corp" `
  -ConnectionSpecificSuffix "corp.michaelwaterman.nl" `
  -RegisterThisConnectionsAddress:$true `
  -UseSuffixWhenRegistering:$true
```

Disable NetBIOS over TCP/IP

In environments where NetBIOS is not required, disabling it reduces legacy name resolution noise and attack surface.

```
Invoke-CimMethod `
  -Query 'SELECT * FROM Win32_NetworkAdapterConfiguration WHERE IPEnabled=1' `
  -MethodName SetTcpipNetbios `
  -Arguments @{ TcpipNetbiosOptions = [uint32]2 }
```

Disable LMHOSTS lookup

Disable LMHOSTS lookup (legacy name resolution behavior):

```
Invoke-CimMethod `
  -ClassName Win32_NetworkAdapterConfiguration `
  -Arguments @{ WINSEnableLMHostsLookup = $false } `
  -MethodName EnableWINS
```

Remove a static IP and switch to DHCP

Enable DHCP on an interface:

```
Set-NetIPInterface -InterfaceAlias "corp" -Dhcp Enabled
```

Note! depending on your starting state, you may also want to remove existing static IPv4 addresses first.

Remove the default gateway

If you want to remove routes / gateway configuration, you can remove non-default routes like this:

```
Get-NetRoute -AddressFamily IPv4 |
Where-Object { $_.NextHop -ne "0.0.0.0" } |
ForEach-Object { Remove-NetRoute -NextHop $_.NextHop -Confirm:$false }
```

IPv6

IPv6 is enabled by default. If your environment explicitly requires disabling it, you can manage bindings per adapter.

List IPv6 binding state:

```
Get-NetAdapterBinding -ComponentID ms_tcpip6
```

Disable IPv6 on a specific adapter:

```
Disable-NetAdapterBinding -Name "corp" -ComponentID ms_tcpip6
```

Disable IPv6 on all adapters:

```
Disable-NetAdapterBinding -Name "*" -ComponentID ms_tcpip6
```

Note! *Disabling IPv6 should be a deliberate decision. In many Microsoft-based environments, IPv6 is expected to be available even when you don't actively use it. My personal advise based on the results of many security audits, turn off IPv6 if you have not configured it on your network, the attack surface is not know by many, yet can easily be misused. And yes this conflicts with Microsoft's statement, although I do agree that organizations should configure IPv6 as well, so turning it of at the host would not be required.*

System identity & domain membership

Once networking is configured and stable, the next step is to define the system's identity. This includes the computer name, time zone, and, in most enterprise environments, domain membership. Doing this in a predictable order helps avoid authentication issues and unnecessary reboots later in the process.

Rename the computer

Set the computer name and reboot the system:

```
Rename-Computer -NewName "<NewName>" -Restart
```

Set the time zone

Time synchronization and correct time zone configuration are critical for Kerberos authentication and logging.

```
Set-TimeZone -Id "W. Europe Standard Time"
```

Tip! You can list available time zones with:

```
Get-TimeZone -ListAvailable
```

Join the domain

Once the system identity is set, join the server to the domain:

```
Add-Computer `
  -DomainName "<DNS Domain Name>" `
  -Credential (Get-Credential) `
  -Restart
```

This command:

- joins the system to the specified domain
- prompts for credentials
- automatically reboots the server

Tip! *In fully automated deployments, credentials are typically handled via secure mechanisms such as deployment accounts, managed identities, or secret stores rather than interactive prompts.*

Windows Firewall

On Windows Server Core, the Windows Firewall is often one of the first things people disable “temporarily”, and then forget about. While that might help during early provisioning or troubleshooting, it should never be the final state of a production server. That said, there *are* moments where temporarily adjusting firewall behavior makes sense, especially during automated builds.

Disable the firewall (all profiles)

To disable the firewall for all profiles:

```
Set-NetFirewallProfile -Profile Domain,Public,Private -Enabled False
```

This is sometimes useful:

- during initial provisioning
- when validating connectivity
- while troubleshooting deployment issues

Enable the firewall (all profiles)

Re-enable the firewall once the system is properly configured:

```
Set-NetFirewallProfile -Profile Domain,Public,Private -Enabled True
```

In most environments, this should be considered the **baseline state**.

Enable ICMPv4 echo reply (ping)

Allow inbound ICMPv4 echo requests (useful for basic connectivity testing):

```
Enable-NetFirewallRule -Name "FPS-ICMP4-ERQ-In"
```

This enables ping responses without opening unnecessary ports or services.

Tip! *Rather than disabling the firewall entirely, it's usually better to explicitly enable only the rules you need. This keeps troubleshooting predictable while maintaining a reasonable security posture.*

Windows Updates

Keeping Windows Server Core up to date is a critical part of any secure baseline. While Windows provides inbox APIs to query updates, update installation is something I prefer to handle in a controlled and scriptable way, especially in automated builds.

Query available updates (inbox API)

Windows exposes update information through a COM-based interface. This allows you to query update metadata locally:

```
$Session = New-Object -ComObject "Microsoft.Update.Session"

$Criteria = "(IsInstalled=0 and DeploymentAction=*) or
             (IsInstalled=1 and DeploymentAction=*) or
             (IsHidden=1 and DeploymentAction=*)"

$Result = $Session.CreateUpdateSearcher().Search($Criteria).Updates

$Result | Select-Object `
    Title,
    @{l='UpdateID';e={$_.Identity.UpdateID}},
    @{l='PublishedDate';e={$_.LastDeploymentChangeTime.ToString('yyyy-MM-dd')}},
    KBArticleIDs,
    IsHidden |
Format-List
```

This provides insight into:

- available updates
- installed updates
- hidden updates
- associated KBs

Remote execution consideration

COM-based update operations ***cannot be executed remotely*** due to security boundaries. If you need to automate this remotely, you'll typically use:

- scheduled tasks
- configuration management tools
- or local execution during provisioning

Installing updates

For installing updates, I use a dedicated PowerShell script, the [Install-WindowsUpdate.ps1](#) script exposes a minimal and explicit parameter set, focused on predictable behavior during automated deployments. It's a simple and to the point script. It can be used but there are far better examples available such as the Windows Update PowerShell module. But hey, this is my take!

Parameter overview

- IncludeDrivers**Includes applicable driver updates during the update run.
- SecurityCriticalOnly**Limits the update selection to security and critical updates only.
- Reboot**Allows the system to reboot if required by installed updates.

User profile management

On long-running servers (or jump hosts), local user profiles can quietly pile up over time. Think temporary admin accounts, vendor logins, or one-time troubleshooting sessions. On Server Core, it's useful to have a quick way to inventory profiles and remove stale ones.

All examples below use Win32_UserProfile via CIM.

Inventory user profiles (Local)

List all local profiles and their paths:

```
Get-CimInstance -ClassName Win32_UserProfile |  
Select-Object LocalPath
```

If you want a more readable overview (path + last use time):

```
Get-CimInstance -ClassName Win32_UserProfile |  
Select-Object LocalPath, LastUseTime, Loaded |  
Sort-Object LastUseTime
```

Tip! If a profile is `Loaded = True`, it's currently in use. Removing loaded profiles will fail (and is a bad idea anyway).

Inventory user profiles (remote)

Query profiles on one or multiple remote servers:

```
Get-CimInstance -ClassName Win32_UserProfile -ComputerName "WINSRV" |  
Select-Object LocalPath
```

Multiple servers:

```
Get-CimInstance -ClassName Win32_UserProfile -ComputerName "SRV1","SRV2","SRV3" |  
Select-Object PSComputerName, LocalPath
```

Remove a specific user profile (Local)

Remove a profile by matching the username at the end of the profile path:

```
Get-CimInstance -ClassName Win32_UserProfile |  
Where-Object { $_.LocalPath.Split('\')[-1] -eq 'UserA' } |  
Remove-CimInstance
```

Remove a specific user profile (remote)

Remove a specific profile from multiple servers:

```
Get-CimInstance -ClassName Win32_UserProfile -ComputerName "SRV1","SRV2","SRV3" |  
Where-Object { $_.LocalPath.Split('\')[-1] -eq 'UserA' } |  
Remove-CimInstance
```

Enable Remote Desktop

On Server Core, Remote Desktop is often required for:

- troubleshooting
- break-glass access
- vendor or support scenarios

Even if you primarily manage servers via PowerShell Remoting or automation, having RDP available (and properly firewalled) can be extremely useful.

Enable Remote Desktop

Enable Remote Desktop by allowing incoming RDP connections:

```
Set-ItemProperty `
  -Path 'HKLM:\System\CurrentControlSet\Control\Terminal Server' `
  -Name "fDenyTSConnections" `
  -Value 0
```

This updates the system setting to allow RDP connections.

Enable the firewall rules for Remote Desktop

Make sure the required firewall rules are enabled:

```
Enable-NetFirewallRule -DisplayGroup "Remote Desktop"
```

This opens TCP port 3389 for inbound RDP traffic, scoped to the active firewall profiles.

Verification

You can verify that Remote Desktop is enabled with:

```
Get-ItemProperty `
  -Path 'HKLM:\System\CurrentControlSet\Control\Terminal Server' `
  -Name "fDenyTSConnections"
```

A value of 0 means Remote Desktop is enabled.

Event Viewer tips (server core)

Even without a graphical Event Viewer, Windows Server Core provides powerful ways to inspect and manage event logs using built-in tooling. During provisioning, troubleshooting, or image preparation, being able to query and clean logs is often more useful than people realize.

List all event logs

To list all available event logs on the system:

This shows both classic and modern (channel-based) logs.

Query recent events from a specific log

For quick troubleshooting, you can query recent events directly:

```
Get-WinEvent -LogName System -MaxEvents 20
```

Or filter by level (for example, errors only):

```
Get-WinEvent -FilterHashtable @{
    LogName = 'System'
    Level    = 2
} -MaxEvents 20
```

Check log size and retention settings

To inspect log configuration details:

This shows:

- maximum log size
- retention behavior
- overwrite settings

Useful when diagnosing missing or truncated logs.

Clear a specific event log

Clear a single log:

This is often useful after:

- troubleshooting sessions
- baseline validation
- pre-handover cleanup

Clear all event logs (final cleanup)

When preparing a system for handover, templating, or final validation, I like to start with a clean slate:

```
wevtutil el | ForEach-Object {
    Write-Host "Clearing $_"
    wevtutil cl "$_"
}
```

This:

- enumerates all event logs
- clears them one by one
- provides clear console feedback

Hint! Use with care. *Clearing all event logs should only be done intentionally, for example during image creation, controlled rebuilds, or before handing a system over*

| to operations.

Active Directory Domain Services (ADDS)

After disk layout, networking, system identity, updates, firewall configuration, and basic operational tooling are in place, the server is finally ready to take on its actual role.

For many environments, that role is *Active Directory Domain Services*. Running ADDS on Windows Server Core is not a compromise, it's a deliberate choice. Fewer components, fewer moving parts, and a smaller attack surface, while still providing the full functionality expected from a domain controller.

Install the ADDS role

Install the required role:

```
Install-WindowsFeature-Domain-Services
```

This installs the binaries but does not yet promote the server.

Prepare the built-in Administrator account

Before promotion, ensure the local Administrator account meets password requirements. From a command prompt:

```
net user administrator /passwordreq:yes
```

This avoids avoidable promotion failures later in the process. This was a problem up to Windows Server 2022, TBH, I've not seen this in Windows Server 2025.

Forest and first Domain Controller

To create a new forest and promote the first domain controller:

```
Install-ADDSForest `
  -CreateDnsDelegation:$false `
  -DatabasePath "E:\NTDS" `
  -DomainMode "WinThreshold" `
  -DomainName "corp.michaelwaterman.nl" `
  -DomainNetbiosName "CORP" `
  -ForestMode "WinThreshold" `
  -InstallDns:$true `
  -LogPath "F:\LOGS" `
  -SysvolPath "D:\SYSVOL" `
  -NoRebootOnCompletion:$false `
  -Force:$true
```

Notes

- NTDS, SYSVOL, and logs are placed on *separate volumes* for performance, recovery, and operational clarity
- DNS is installed and integrated automatically
- The system will reboot as part of the promotion

Additional Domain Controller

To add an additional domain controller to an existing domain:

```
Install-ADDSDomainController `
  -DomainName "corp.michaelwaterman.nl" `
  -CreateDnsDelegation:$false `
  -DatabasePath "E:\NTDS" `
  -InstallDns:$true `
  -SysvolPath "D:\SYSVOL" `
  -LogPath "F:\LOGS" `
  -NoRebootOnCompletion:$false `
  -Force:$true `
  -Credential (Get-Credential)
```

This promotes the server and joins it to the existing domain in one operation.

Automation perspective

ADDS promotion is one of those tasks that looks complex, but is actually very automation-friendly when:

- prerequisites are handled upfront
- paths are explicit
- reboots are expected and controlled

On Server Core, this process is often *more reliable* than on GUI-based systems, simply because there is less that can interfere.

Administrative Tools (RSAT) – completeness note

For completeness, it's worth mentioning Remote Server Administration Tools (RSAT) when working with Active Directory. These tools provide graphical and command-line management capabilities for roles such as:

- Active Directory Users and Computers
- Active Directory Administrative Center
- DNS Management
- Group Policy Management

However, *these tools do not belong on a domain controller*. In modern, security-focused environments, administrative tooling should be installed on a *Privileged Access Workstation (PAW)* or dedicated management system — not on the servers hosting the roles themselves.

Installing RSAT (reference only)

For reference, RSAT components can be installed using:

```
Get-WindowsCapability -Name RSAT* -Online | Add-WindowsCapability -Online
```

Important! *Installing RSAT directly on a domain controller increases the attack surface and goes against the principle of role separation.*

*This command is shown **for completeness only** and should be executed on a PAW or management workstation, not on the domain controller itself.*

Features on Demand (FoD)

Installing RSAT via Windows Update can take a significant amount of time and may depend on internet connectivity. In controlled environments, it's often preferable to install RSAT from a Features on Demand (FoD) ISO:

```
Get-WindowsCapability -Name RSAT* -Online |  
Add-WindowsCapability -Online `  
-LimitAccess `  
-Source "C:\FOD-ISO-Extracted\"
```

For offline installation of RSAT features (for PAWs or management workstations), you can use the [Features on Demand ISO](#) matching your Windows version. Microsoft documentation also [explains](#) how to use this ISO to install optional features offline:

If you need a fallback RSAT download (older standalone package), Microsoft's official RSAT download page is here: <https://www.microsoft.com/nl-nl/download/details.aspx?id=45520>

Security perspective

Separating administrative tooling from role hosting:

- reduces lateral movement opportunities
- limits credential exposure
- aligns with tiered administration models

Active Directory should be *managed remotely*, not locally.

A better PowerShell experience on Server Core

Before we wrap up, it's worth touching on the PowerShell experience itself. Server Core is minimal by design, but that doesn't mean the shell has to feel minimal. Earlier this year I wrote a short piece on improving the interactive PowerShell experience on Server Core:

A Better PowerShell Experience on Windows Server Core

<https://michaelwaterman.nl/2025/11/09/a-better-powershell-experience-on-windows-server-core/>

That post covers:

- upgrading PowerShell hosting on Core
- optional addons for better console usability
- tips to make command entry and navigation less painful

If you've ever found the default console limiting, that post is a great companion to this one, especially when you're spending most of your time executing and automating with PowerShell.

Server Core app compatibility feature on demand

For completeness, it's worth mentioning the *Server Core App Compatibility Feature on Demand*. This is an optional feature provided by Microsoft that adds a limited set of graphical components to Windows Server Core. This feature exists to improve application and tooling compatibility, not to turn Server Core into a full GUI-based server.

What does it add?

When installed, the App Compatibility FoD adds support for a small number of graphical and MMC-based tools, including:

- Event Viewer (`eventvwr.msc`)
- Device Manager
- Performance Monitor
- Limited MMC framework support
- A minimal File Explorer (without the full Windows shell)

This can be useful in scenarios where:

- legacy tooling requires MMC
- troubleshooting must be done locally
- a temporary transition from Full GUI to Core is in progress

What it does *not* do

It's equally important to understand what this feature does **not** provide:

- No full Windows Explorer shell
- No replacement for RSAT on a PAW
- No justification for managing AD locally on a domain controller

Server Core remains Server Core, just with a small compatibility layer added.

Installation

The feature can be installed using PowerShell:

```
Add-WindowsCapability -Online -Name ServerCore.AppCompatibility~~~~0.0.1.0
```

A reboot is required after installation.

In conclusion

Windows Server Core is often perceived as “*harder*” to work with simply because it removes the comfort of a graphical interface. In reality, it forces a healthier mindset: be explicit, be predictable, and automate everything. Throughout this post, we configured a Server Core system end-to-end using nothing but PowerShell. From disks and networking to system identity, updates, firewall rules, operational tooling, and finally Active Directory itself. No clicks, no wizards, no guesswork.

That's not a limitation, it's a strength.

When you treat Server Core as something to *provision* rather than *configure*, a few things naturally fall into place:

- deployments become repeatable
- changes become intentional
- troubleshooting becomes easier
- and the overall attack surface stays smaller

Not every environment will look exactly the same, and that's fine. The commands and patterns shown here are not meant to be copy-pasted blindly, but adapted to fit your standards, automation pipelines, and security model. If this post nudges you away from manual configuration and towards scripted, predictable builds, even just a little, then it has done its job.

Make automation boring.

Make it predictable.

Make Automation Great Again.

As always, if you have any question or remarks, let me know. Until next time.